

Predictive Data Analytics with Python

Unit 4

Analytics Types

- ▶ Predictive Analytics
- ▶ Descriptive Analytics
- ▶ Prescriptive Analytics

Predictive analytics

- ▶ Predictive analytics is a branch of data analytics that uses historical data, statistical algorithms, and machine learning techniques to identify patterns and predict future outcomes.
- ▶ It helps organizations make data-driven decisions by estimating what is likely to happen based on past trends and correlations.

Applications of Predictive Analytics

- ▶ **Credit Scoring:** Banks use predictive analytics to assess a customer's creditworthiness. By analyzing past transaction data, payment history, and credit scores, banks predict whether a customer is likely to default on a loan.
- ▶ **Fraud Detection:** Machine learning models analyze transaction behaviors and flag unusual activity that may indicate fraud.

Descriptive Analytics

- ▶ Descriptive analytics is the process of analyzing historical data to understand trends, patterns, and insights. It helps organizations **summarize past events using data aggregation, reporting, and visualization techniques.**
- ▶ Unlike predictive analytics, which forecasts future trends, descriptive analytics focuses on what has already happened.

Applications

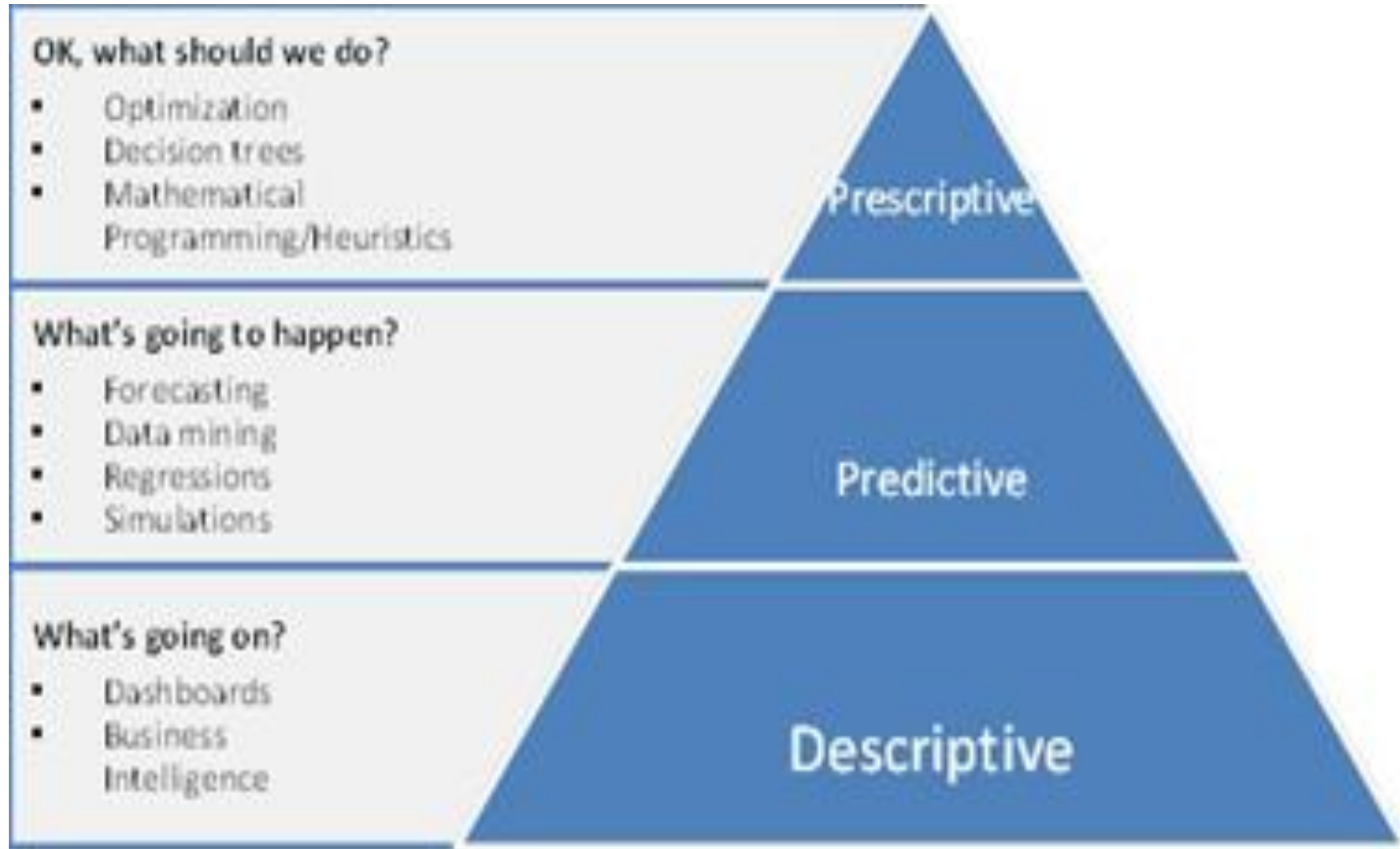
- ▶ **Sales Performance Analysis:** An online retailer analyzes past sales data to identify which products sold the most in the last quarter. The analysis may include revenue trends, customer demographics, and seasonal buying patterns, helping the retailer make informed business decisions.
- ▶ **Website Traffic Reports:** Google Analytics provides website owners with reports on user visits, page views, bounce rates, and session durations, helping them understand user behavior on their platform.

Prescriptive Analytics

- ▶ Prescriptive analytics is the most advanced type of data analytics that goes beyond predicting future outcomes by recommending actions to achieve the best possible result.
- ▶ It uses machine learning, artificial intelligence, and optimization algorithms to suggest decision-making strategies.

Applications

- ▶ **Personalized Treatment Plans:** Hospitals use prescriptive analytics to recommend the best treatment for a patient based on their medical history, genetic profile, and current symptoms.
- ▶ AI-driven models analyze vast amounts of patient data to suggest the most effective treatment while minimizing risks.



Association Rules

- ▶ Association Rule Mining is a technique in data mining used to discover relationships or patterns between items in large datasets.
- ▶ It identifies how frequently items appear together in a dataset and is commonly used in market basket analysis, recommendation systems, and fraud detection.

Example of Association Rule

- ▶ A classic example is **Market Basket Analysis** in retail:
- ▶ If many customers who buy **bread** also buy **butter**, the association rule can be:

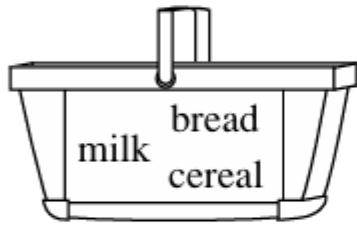
{Bread} → {Butter} (If a customer buys bread, they are likely to buy butter).

Retailers use this insight to place these products closer together or offer discounts on related items.

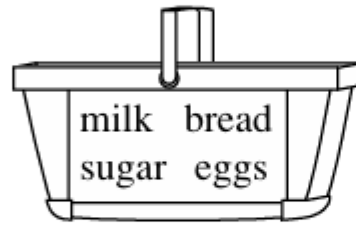


Which items are frequently purchased together by customers?

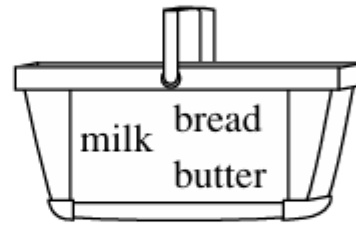
Shopping Baskets



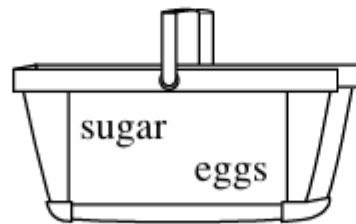
Customer 1



Customer 2



Customer 3



Customer n

Market Analyst

Market basket analysis.

Measures of Association Rule Mining

- ▶ Association Rule Mining uses three key measures to evaluate the strength and usefulness of discovered rules:
1. Support: Measures how frequently an itemset appears in the dataset.

$$\text{Support}(A \rightarrow B) = \frac{\text{Transactions Containing } A \text{ and } B}{\text{Total Transactions}}$$

Example of Support

- ▶ If 100 out of 1,000 transactions contain both milk and bread, then:

$$\text{Support}(\{\text{milk}, \text{bread}\}) = \frac{100}{1000} = 10\%$$

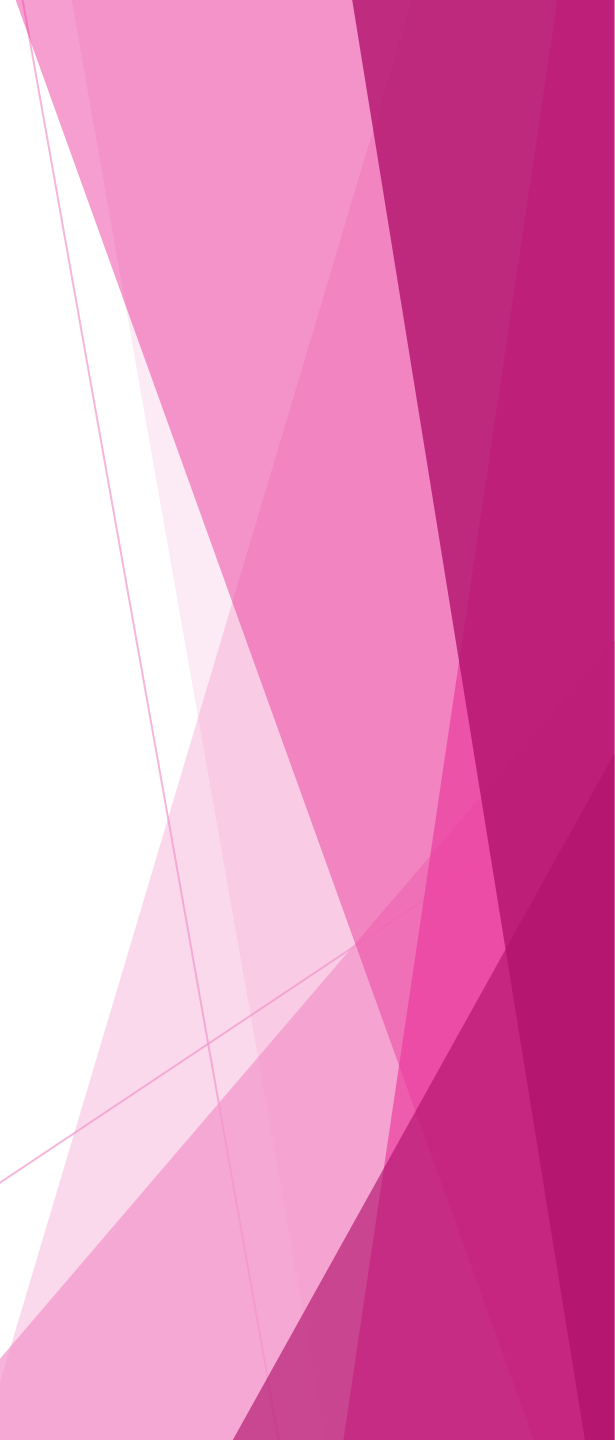
Confidence

- Measures how often the rule $A \rightarrow B$ is found to be true.

$$\text{Confidence}(A \Rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

Example: If 200 transactions contain bread, and out of these, 100 also contain milk

$$\text{Confidence}(\{Bread\} \Rightarrow \{Milk\}) = \frac{100}{200} = 50\%$$

- 
- ▶ In general, association rule mining can be viewed as a two-step process:
 1. Find all frequent itemsets: By definition, each of these item sets will occur at least as frequently as a predetermined minimum support count, minsup.
 2. Generate strong association rules from the frequent itemsets: By definition, these rules must satisfy minimum support and minimum confidence.

Frequent Itemset Mining Methods

- ▶ Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation
 1. Apriori is a seminal algorithm proposed by R. Agrawal and R. Srikant in 1994 for mining frequent itemsets for Boolean association rules.
 2. The name of the algorithm is based on the fact that the algorithm uses prior knowledge of frequent itemset properties
 3. Apriori employs an iterative approach known as a level-wise search, where k -itemsets are used to explore $k+1$ itemsets.

Apriori Steps

► Step 1: Set Minimum Support Threshold

- Define a **minimum support** value to filter out infrequent itemsets.
- Support of an itemset = (Number of transactions containing the itemset) / (Total transactions).

► Step 2: Generate Frequent 1-Itemsets (L1)

- Scan the dataset and count occurrences of each item.
- Remove items that do not meet the minimum support threshold.
- The remaining frequent 1-itemsets form **L1**.

► **Step 3: Generate Candidate Itemsets (C_k) for $k \geq 2$**

- Use the **Apriori property** to generate candidate **k-itemsets** from frequent **(k-1)-itemsets**.
- Candidate itemsets are formed by joining $L(k-1) \times L(k-1)$ (self-joining step).

► **Step 4: Prune Infrequent Candidates**

- Remove itemsets that have infrequent subsets.
- Count the occurrences of each candidate itemset in transactions.
- Keep only those that meet the minimum support threshold → These form **L_k** .

► **Step 5: Repeat Until No More Frequent Itemsets**

- Continue generating **$L_2, L_3, \dots, L_k$** until no more frequent itemsets can be found.

► **Step 6: Generate Association Rules**

- Use frequent itemsets to generate **association rules**.

Example

Transactional Data for an *AllElectronics* Branch

<i>TID</i>	<i>List of item_IDs</i>
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

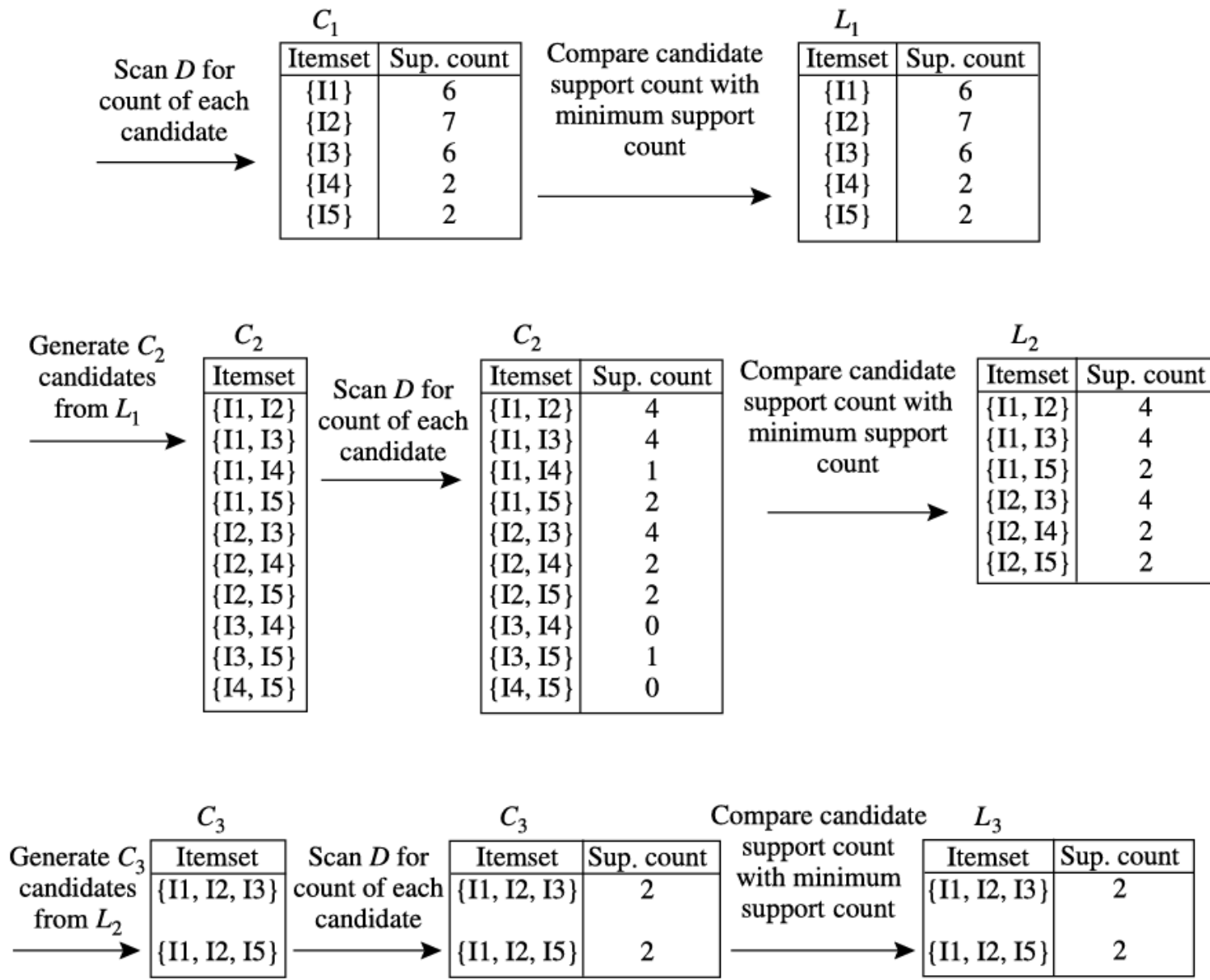


Figure 6.2 Generation of the candidate itemsets and frequent itemsets, where the minimum support count is 2.

FP Growth Method

- ▶ Limitations of Apriori Method:
- ▶ It may still need to generate a huge number of candidate sets
- ▶ It may need to repeatedly scan the whole database and check a large set of candidates by pattern matching.
- ▶ It is costly to go over each transaction in the database to determine the support of the candidate itemsets.

FP Growth Method

- ▶ Can we design a method that mines the complete set of frequent itemsets without such a costly candidate generation process?”
- ▶ An interesting method in this attempt is called frequent pattern growth, or simply FP-growth, which adopts a divide-and-conquer strategy

Steps of the FP-Growth Algorithm

► Step 1: Scan the Dataset and Count Item Frequencies

- Scan the dataset once and count the frequency of each item.
- Remove infrequent items (those below the minimum support threshold).
- Sort the remaining items in descending order of frequency.

► Step 2: Build the FP-Tree

- The root of the tree is a null node.
- Insert transactions into the FP-Tree, maintaining the item order determined in Step 1.
- If a transaction shares a prefix with an existing path, increase the count of the existing nodes; otherwise, create a new branch.

► Step 3: Generate Conditional Pattern Bases

- Extract frequent itemsets by traversing the FP-Tree.
- For each item (starting from the least frequent), construct a **conditional pattern base**, which is a sub-database containing prefix paths leading to that item.

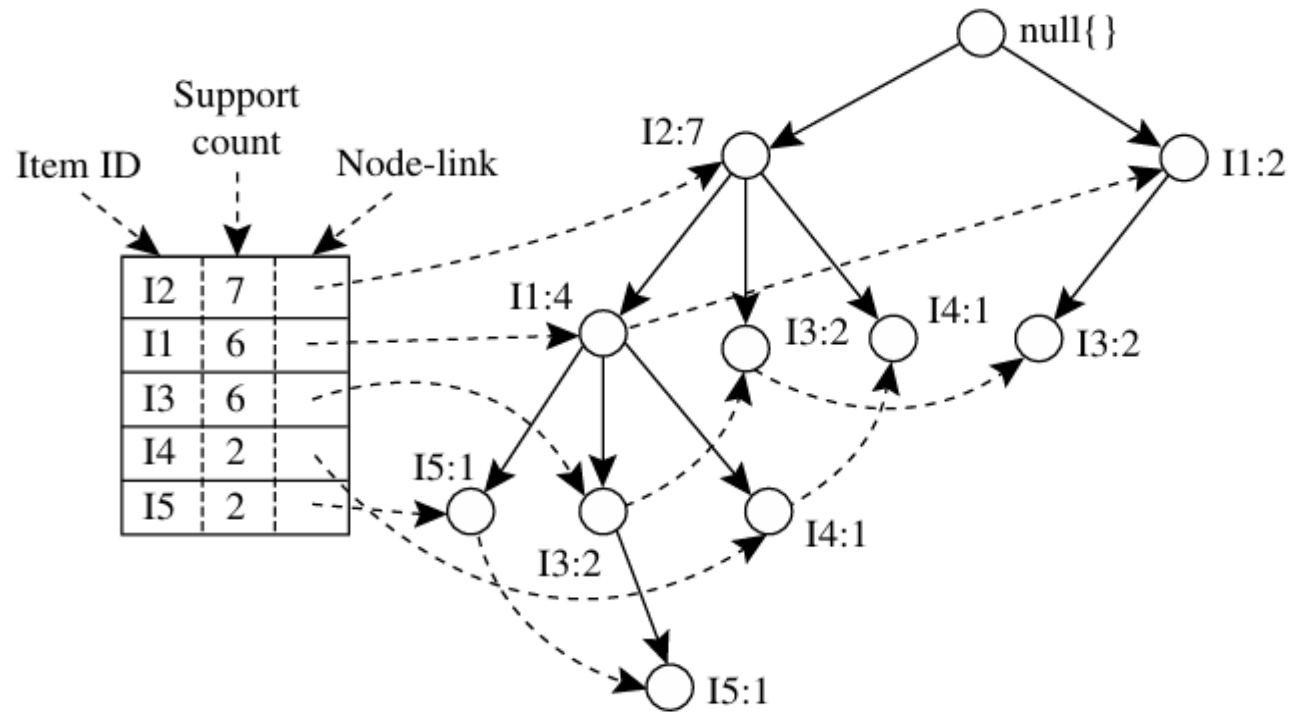
► Step 4: Construct Conditional FP-Trees

- Using the conditional pattern base, build a new FP-Tree for the item.
- Recursively apply the FP-Growth algorithm to find frequent itemsets.

► Step 5: Extract Frequent Itemsets

- From the conditional FP-Trees, combine the item with prefix paths to generate frequent itemsets.
- Continue recursively until all frequent patterns are found.

Example



re 6.7 An FP-tree registers compressed, frequent pattern information.

Table 6.2 Mining the FP-Tree by Creating Conditional (Sub-)Pattern Bases

<i>Item</i>	<i>Conditional Pattern Base</i>	<i>Conditional FP-tree</i>	<i>Frequent Patterns Generated</i>
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	$\langle I2: 2, I1: 2 \rangle$	{I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}
I4	{{I2, I1: 1}, {I2: 1}}	$\langle I2: 2 \rangle$	{I2, I4: 2}
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	$\langle I2: 4, I1: 2 \rangle, \langle I1: 2 \rangle$	{I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2}
I1	{{I2: 4}}	$\langle I2: 4 \rangle$	{I2, I1: 4}

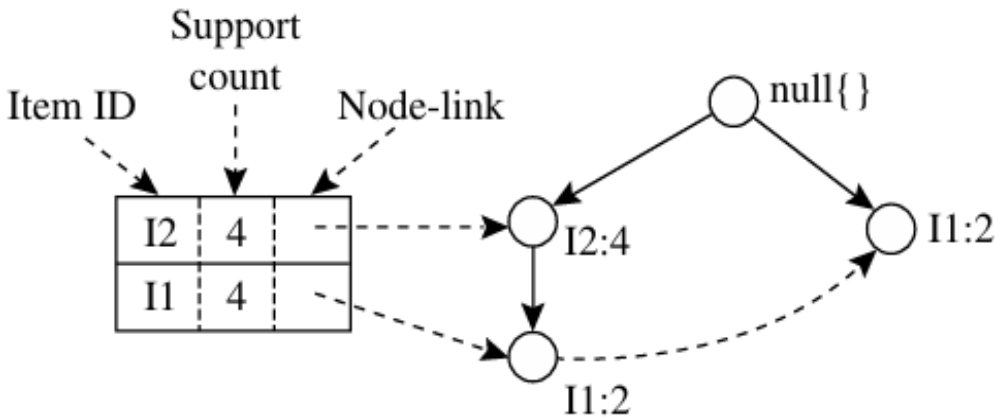


Figure 6.8 The conditional FP-tree associated with the conditional node I3.

Regression

- ▶ Regression is a statistical and machine learning technique used to model the relationship between a dependent variable (target/output) and one or more independent variables (features/inputs).
- ▶ It helps in predicting continuous values based on input data.

Types of Regression

► Linear Regression

- Models the relationship using a straight line.
- Example: Predicting house prices based on area size.
- Equation: $Y=mx+c$
- X- area size
- Y=house price
- m is the slope (how much y changes for each unit increase in x)
- c is the intercept (where the line crosses the y -axis)

Linear Regression

- ▶ A **line in linear regression** is just a straight line that best describes the relationship between two things.
- ▶ For example, imagine you are looking at **how study hours affect exam scores**. If we plot students on a graph with:
 - ▶ **Study hours on the x-axis**
 - ▶ **Exam scores on the y-axis**
- ▶ The data points will be scattered, but there will be a **general trend**—students who study more tend to score higher.
- ▶ **The Regression Line**
- ▶ Now, imagine drawing a **single straight line** that goes **through the middle of all these points**. This line shows the overall trend:
 - ▶ If the line goes **upward**, it means more study time leads to higher scores.
 - ▶ If the line is **flat**, it means study time doesn't affect scores much.
 - ▶ If the line goes **down**, it would mean studying more actually lowers scores (which is unlikely in this case).

Linear Regression

- ▶ Let's say you have a **simple linear equation** for predicting a salary:
- ▶ $\text{Salary} = 5000 \times \text{Years of Experience} + 30000$
- ▶ The **intercept (30,000)** means that even if someone has **0 years of experience**, they still earn **\$30,000**.
- ▶ The **slope (5000)** means that for **every additional year of experience**, the salary increases by **\$5,000**.

- ▶ **Multiple Linear Regression:** Extends linear regression to multiple independent variables.
- ▶ **Logistic Regression** (*for classification problems, not strictly regression*)
 - ▶ Models the probability of a categorical outcome (e.g., yes/no).
 - ▶ Example: Spam email detection.

Steps in Linear Regression

- ▶ Linear Regression is a supervised learning algorithm used to model the relationship between a dependent variable (target) and one or more independent variables (features).
- ▶ The goal is to fit a straight line that best represents the data. Below are the key steps involved:

Step 1: Define the Problem

Identify the dependent variable (YYY) and independent variable(s) (XXX).

Determine whether a linear relationship exists between the variables.

- ▶ **Example:** Predicting house prices based on size.
- ▶ $Y = \text{Price}$, $X = \text{Size}$

Step 2: Collect and Preprocess Data

- **Gather data:** Collect relevant data points.
- **Handle missing values:** Impute or remove missing data.
- **Remove outliers:** Use statistical methods like Z-score or IQR.
- **Encode categorical variables (if any):** Convert categories into numerical values.
- **Normalize or standardize data (if required):** Helps with model convergence.

Step 3: Split Data into Training and Testing Sets

- ▶ Typically, split data into **80% training** and **20% testing** using `train_test_split()` in python
- ▶ Training set is used to build the model.
- ▶ Testing set is used to evaluate the model's performance.

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 4: Fit the Model (Train the Algorithm)

- ▶ Linear regression follows the equation:

$$Y=mX+c$$

- ▶ For multiple variables, it extends to:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

where:

- Y = dependent variable
- X = independent variable(s)
- β_0 = intercept (constant term)
- $\beta_1, \beta_2, \dots, \beta_n$ = regression coefficients (slopes)

In python, code is

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Step 5: Evaluate the Model

- ▶ After training, assess model performance using metrics

1. R-squared (R^2)

- Measures how well the independent variables explain the variance in the dependent variable.
- R^2 close to 1 → Good fit; close to 0 → Poor fit.

```
r2_score = model.score(X_test, y_test)
```

Step 5: Evaluate the Model

- ▶ Imagine you're trying to predict a student's **exam score** based on the number of hours they study.
- ▶ **A high R^2 (close to 1 or 100%) means:**
→ **Your prediction is very accurate**—most of the differences in students scores can be explained by study time.
- ▶ **Low R^2 (e.g., 0.2)**
→ **Your prediction is weak**—study time explains very little about why students score higher or lower.
- ▶ **Negative R^2** → Your prediction is even **worse than randomly guessing the average score**.

► Mean Squared Error (MSE)

2. Mean Squared Error (MSE)

- Measures the average squared differences between actual and predicted values.

$$MSE = \frac{1}{n} \sum (Y_i - \hat{Y}_i)^2$$

```
from sklearn.metrics import mean_squared_error  
y_pred = model.predict(X_test)  
mse = mean_squared_error(y_test, y_pred)
```

3. Mean Absolute Error (MAE)

- Measures the absolute average error.

$$MAE = \frac{1}{n} \sum |Y_i - \hat{Y}_i|$$

```
from sklearn.metrics import mean_absolute_error  
mae = mean_absolute_error(y_test, y_pred)
```

Step 6: Interpret the Results

- ▶ **Check coefficients (B):** Analyze feature importance.
- ▶ **Analyze residuals:** If residuals show a pattern, the model may not be suitable.
- ▶ **Check for multicollinearity** (for multiple regression): Use Variance Inflation Factor (VIF).

```
import pandas as pd  
coef_df = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_})  
print(coef_df)
```

Step 7: Make Predictions

- Once the model is validated, use it for new data predictions:

```
new_data = [[2000]] # Example: House size 2000 sq. ft  
predicted_price = model.predict(new_data)
```

Step 8: Optimize the Model (if needed)

- ▶ Add more features if performance is low.
- ▶ Transform features (e.g., log transformation for skewed data).
- ▶ Apply regularization (Ridge/Lasso) if overfitting is observed.
- ▶ Use polynomial regression if data shows non-linearity.

Conclusion of Linear Regression

- ▶ Linear Regression is simple yet powerful for modeling relationships between variables. However, it works best when:
- ▶ Data is **linearly related**
 - ✓ No multicollinearity in multiple regression
 - ✓ No high variance (overfitting)

Multicollinearity in Regression Model

- ▶ **Multicollinearity** occurs when two or more independent variables in a regression model are highly correlated, meaning one can be predicted from another.
- ▶ This can make it difficult to determine the effect of each variable on the dependent variable, leading to unreliable coefficient estimates.

Example Scenario

- ▶ Let's say we are building a model to predict **house prices** based on:
 1. **House Size (sq. ft.)**
 2. **Number of Bedrooms**
 3. **Number of Bathrooms**
- ▶ Usually, **House Size** and **Number of Bedrooms** are highly correlated because larger houses tend to have more bedrooms. I
- ▶ If both variables are included in the regression model, it may cause multicollinearity

Effects of Multicollinearity

- ▶ coefficients (β) become unstable and change significantly with small data modifications.
- ▶ Model interpretation becomes difficult.
- ▶ Standard errors of coefficients increase, leading to statistically insignificant results.

Detecting Multicollinearity

- ▶ A simple way to detect multicollinearity is by checking the correlation between independent variables.

```
import pandas as pd
import seaborn as sns

# Sample data
data = {
    'House_Size': [1500, 1800, 2200, 2500, 3000],
    'Bedrooms': [3, 4, 4, 5, 6],
    'Bathrooms': [2, 2, 3, 3, 4]
}

df = pd.DataFrame(data)

# Compute correlation matrix
sns.heatmap(df.corr(),annot=True)
```

Look for correlations above 0.7 or below -0.7 between independent variables.



2. Variance Inflation Factor (VIF)

VIF measures how much the variance of a regression coefficient is inflated due to multicollinearity.

$$VIF = \frac{1}{1 - R^2}$$

- VIF < 5: No serious multicollinearity.
 - VIF > 10: High multicollinearity, needs attention.
- **R^2 = Coefficient of determination**, which measures how well a variable is explained by the other independent variables. **Higher R^2 means higher multicollinearity**, making it harder to determine how each predictor affects the outcome.

Fixing Multicollinearity

- ▶ **Remove One of the Correlated Features**

If **House Size** and **Bedrooms** are highly correlated, remove one of them.

- ▶ **Use Principal Component Analysis (PCA)**

- Reduces dimensions while retaining variance.

Logistic Regression

- ▶ **Logistic Regression** is a supervised learning algorithm used for **classification problems** (e.g., spam detection, disease prediction). It predicts the probability of an outcome being in a certain class (e.g., 0 or 1).
- ▶ Unlike **Linear Regression**, which predicts continuous values, Logistic Regression uses the **sigmoid function** to map predictions between 0 and 1:

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}}$$

1. Define the Problem

- ▶ Identify the **dependent variable** (binary target: Yes/No, 1/0).
- ▶ Identify **independent variables** (features affecting the outcome).
- ▶ **Example:** Predicting whether a student passes (1) or fails (0) based on **study hours** and **attendance**.

2. Collect and Preprocess Data

- ▶ **Handle missing values:** Remove or impute missing data
- ▶ **Convert categorical variables:** Use **One-Hot Encoding** if needed.
- ▶ **Standardize numerical variables:** Scaling improves model performance.
- ▶ **Split data:** 80% for training, 20% for testing.

```
▶ from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```


3. Fit the Logistic Regression Model

- ▶ Train the model using Maximum Likelihood Estimation (MLE), which finds the best coefficients.

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

4. Make Predictions

- Predict probabilities and classify observations.

```
y_pred_prob = model.predict_proba(X_test)[: , 1] # Probability of class 1  
y_pred = model.predict(X_test) # Final classification (0 or 1)
```

5. Evaluate Model Performance

a) Accuracy Score

- Measures the percentage of correctly classified instances.

```
from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy:", accuracy)
```

B) Confusion Matrix

- Shows True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

```
from sklearn.metrics import confusion_matrix  
  
cm = confusion_matrix(y_test, y_pred)  
print("Confusion Matrix:\n", cm)
```

c) Precision, Recall, F1-score

- Precision: $TP / (TP + FP)$
- Recall: $TP / (TP + FN)$
- F1-score: Harmonic mean of precision & recall.
$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
- Precision tells us **how accurate the positive predictions are**. High precision means fewer false positives.
- Recall tells Out of all the real positive cases, how many did the model detect?
- Use **high precision** when **false positives are costly** (e.g., you don't want to mistakenly arrest innocent people in fraud detection).
- Use **high recall** when **false negatives are risky** (e.g., medical diagnosis—missing a serious disease can be life-threatening).

```
from sklearn.metrics import classification_report  
  
print(classification_report(y_test, y_pred))
```

6. Interpret Model Coefficients

- ▶ **Positive coefficient** → Increases probability of class 1. (As X increases, Y also increases)
- ▶ **Negative coefficient** → Decreases probability of class 1.

```
import pandas as pd

coef_df = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_[0]})
print(coef_df)
```

7. Optimize the Model (If Needed)

- ▶ Handle class imbalance(sometimes one class has way more samples than another (e.g., 95% of data belongs to Class A, and only 5% to Class B))
- ▶ **Feature selection** (Remove irrelevant features).
- ▶ **Regularization** (L1 or L2 penalty to reduce overfitting).

```
# Apply L1 (Lasso) regularization  
model = LogisticRegression(penalty='l1', solver='liblinear')  
model.fit(X_train, y_train)
```

Conclusion of Logistic Regression

- ▶ Logistic Regression is a simple yet powerful classification algorithm.
- ▶ Works best when **data is linearly separable**.
- ▶ Requires **balanced class distribution** for optimal performance.

Regularization

- ▶ Imagine you are preparing for an exam. You have a textbook with 500 pages, but only **50 pages are really important** for the test.
- ▶ If you try to **memorize everything** (all 500 pages), you might get confused during the exam. (**Overfitting**)
- ▶ If you focus only on the **important 50 pages**, you will understand the key concepts and do well on the test. (**Generalization**)
- ▶ Regularization helps your model focus on the most important patterns instead of memorizing everything.

Regularization

- ▶ When training a model, it learns **weights (coefficients)** for different features. Some weights might be too large, making the model too complex.
- ▶ **Regularization adds a small penalty to these weights to keep them small and simple.**
- ▶ There are two main types:
- ▶ **L1 Regularization (Lasso Regression)**
 - ▶ Shrinks some feature weights to **exactly zero** (removes unimportant features).
 - ▶ Example: If you are studying for an exam, this is like **ignoring useless topics**.
- ▶ **L2 Regularization (Ridge Regression)**
 - ▶ Shrinks all feature weights but **doesn't make them zero** (reduces their impact).
 - ▶ Example: Instead of memorizing every page, you **just skim less important topics**.

Regularization

- ▶ **Why is Regularization Important?**
- ▶ Prevents **overfitting** (model doesn't memorize training data).
- ▶ Improves **generalization** (model works better on new data).

Classification

- ▶ **Classification** is a **supervised learning** task in machine learning where the goal is to categorize data into predefined classes or labels.
- ▶ The model learns patterns from labeled training data and predicts the class for new, unseen data.

Types of Classification

1. Binary Classification

Two possible classes (e.g., 0 or 1, Yes or No)

- **Example:** Spam Detection (Spam or Not Spam).
- Disease Diagnosis (Diseased or Healthy).
- Loan Approval (Approved or Rejected).

2. Multi-Class Classification

- ▶ More than two classes
- **Example:** Handwritten Digit Recognition (0-9).
- Sentiment Analysis (Positive, Negative, Neutral).
- Animal Classification (Dog, Cat, Bird).

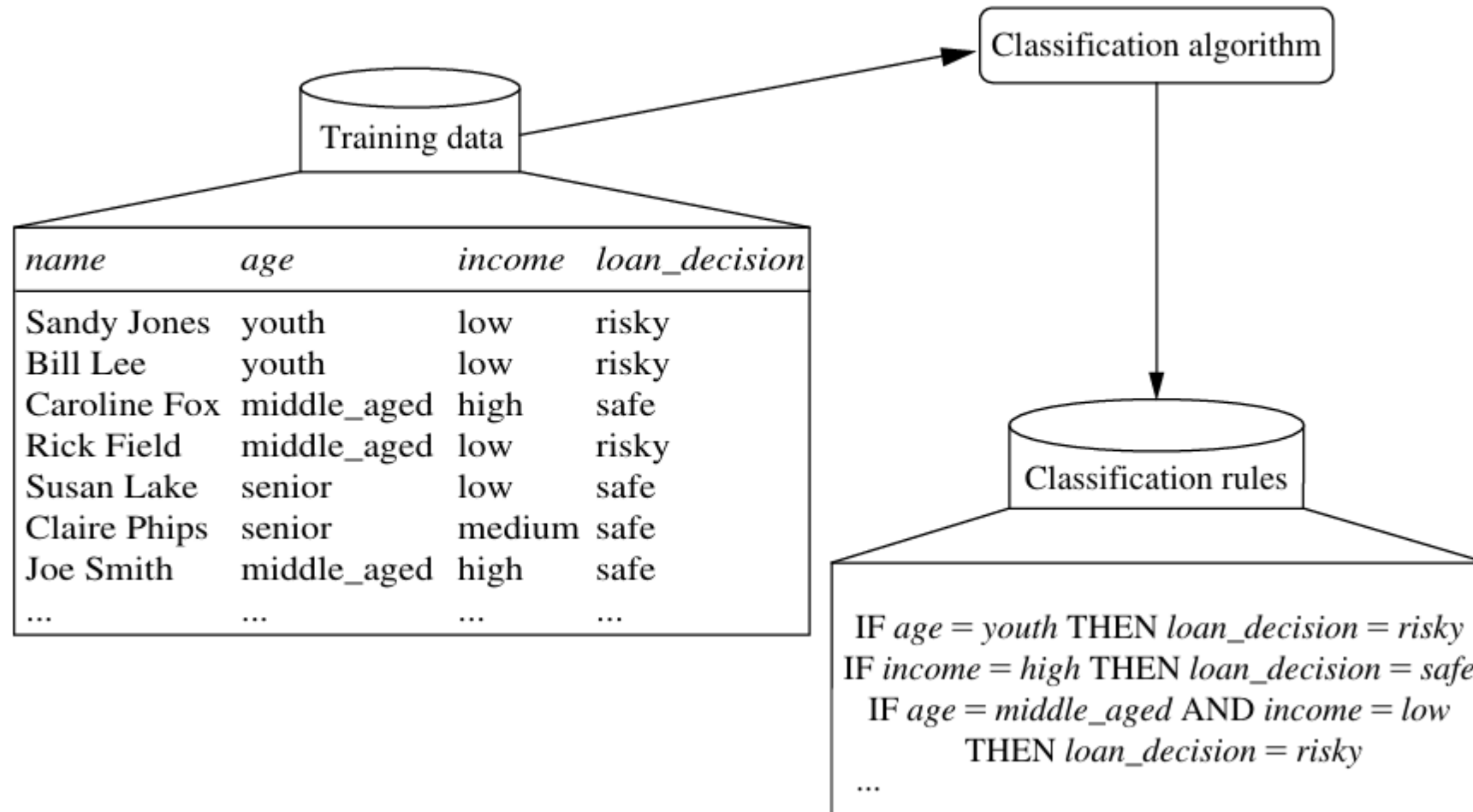
3. Multi-Label Classification

Each sample can belong to multiple classes simultaneously.

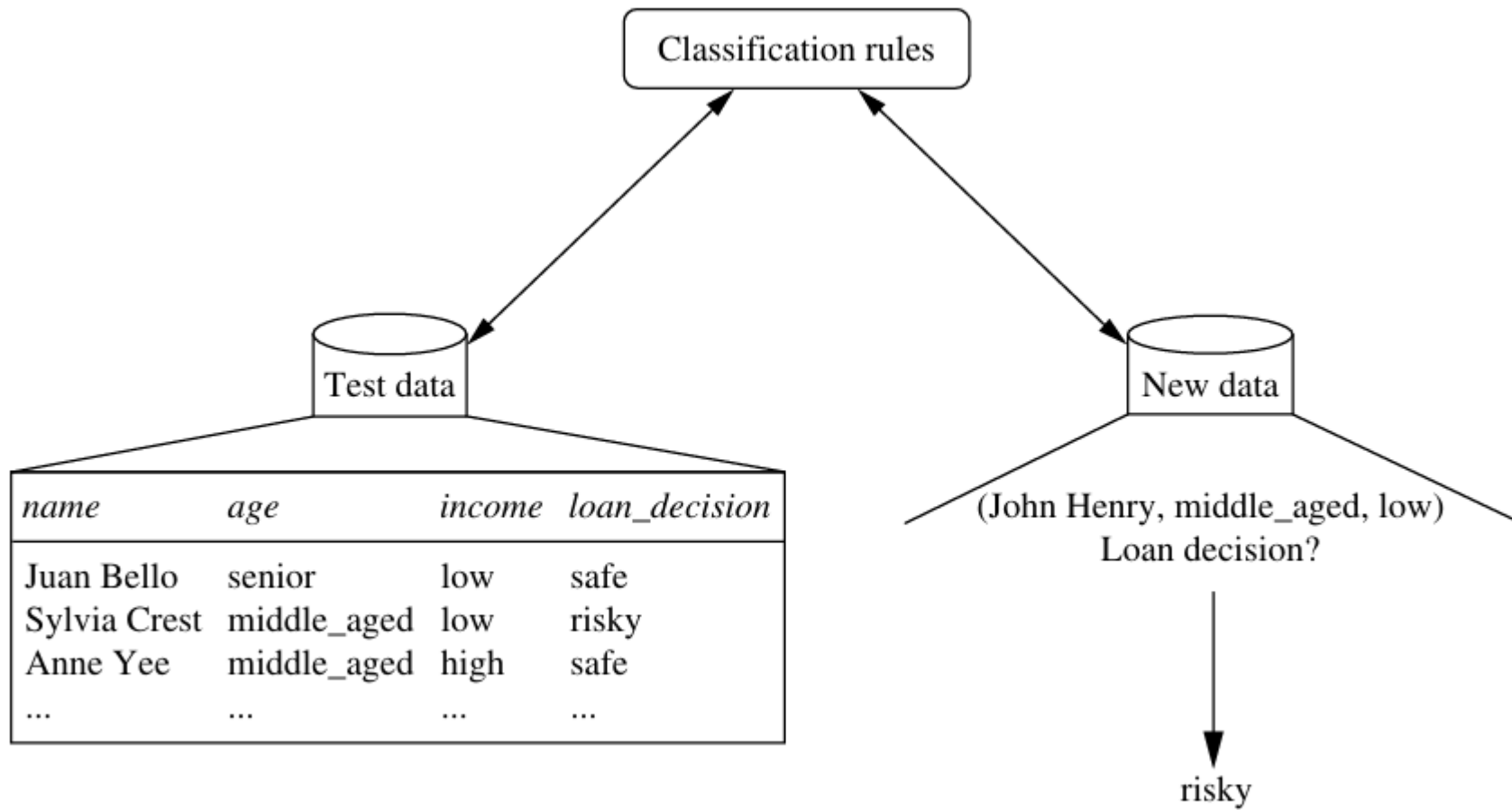
- **Example:** Movie Genre Classification (A movie can be **Action + Comedy**).
- Object Detection (An image can contain **Car + Person + Tree**).

Classification steps

- ▶ The data classification process:
- ▶ (a) **Learning**: Training data are analyzed by a classification algorithm. Here, the class label attribute is loan decision, and the learned model or classifier is represented in the form of classification rules.
- ▶ (b) **Classification**: Test data are used to estimate the accuracy of the classification rules. If the accuracy is considered acceptable, the rules can be applied to the classification of new data tuples.



(a)



(b)

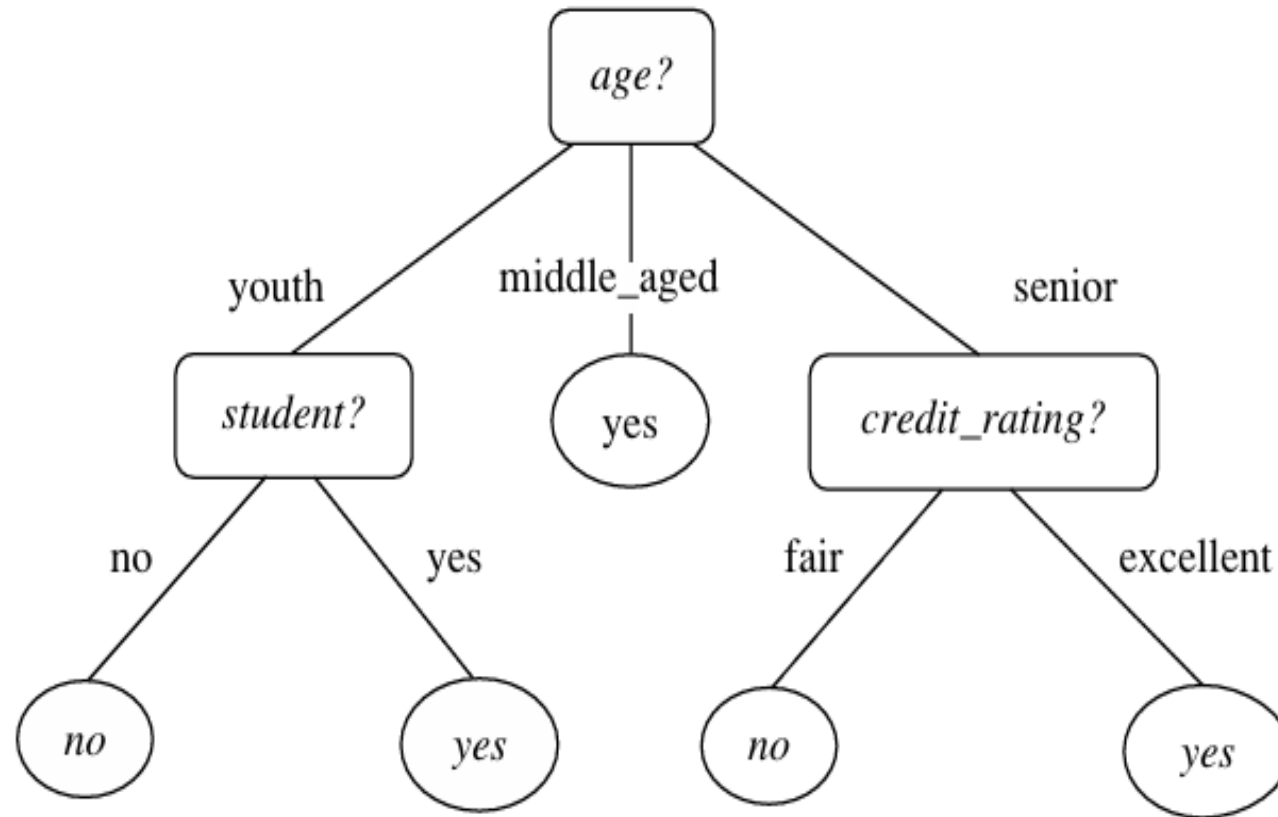
Common Algorithms

Algorithm	Type	Example Use Case
Logistic Regression	Binary	Fraud detection
Decision Tree	Binary/Multi	Customer segmentation
Random Forest	Binary/Multi	Disease prediction
Support Vector Machine (SVM)	Binary/Multi	Email spam filtering
K-Nearest Neighbors (KNN)	Binary/Multi	Recommender systems
Naïve Bayes	Binary/Multi	Sentiment analysis
Neural Networks (Deep Learning)	Multi-label	Image recognition

Decision Tree Induction

- ▶ Decision tree induction is the learning of decision trees from class-labeled training tuples.
- ▶ A decision tree is a flowchart-like tree structure, where each internal node (non leaf node) denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (or terminal node) holds a class label.
- ▶ The topmost node in a tree is the root node

Example



Steps in Decision Tree Classification

- ▶ A **Decision Tree** is a supervised learning algorithm used for both **classification** and **regression** tasks.
- ▶ It works by splitting data into smaller subsets based on feature values, creating a tree-like structure where each node represents a decision.

1. Collect and Prepare Data

- ▶ Ensure the dataset contains both **features (X)** and **labels (Y)**.
- ▶ Example dataset: **Predicting whether a customer will buy a product (Yes/No)**.

2. Preprocess the Data

- ▶ **Handle missing values** (imputation or removal).
- ▶ **Convert categorical variables** into numerical form (One-Hot Encoding)
- ▶ **Scale numerical features** if necessary
- ▶ **Split data** into training and testing sets (80-20 split)

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Build the Decision Tree Model

- ▶ Train a decision tree using the **Gini Index** or **Entropy** as the splitting criterion.
- ▶ The tree recursively splits based on the **best feature** until a stopping condition is met.

```
from sklearn.tree import DecisionTreeClassifier

# Initialize model
model = DecisionTreeClassifier(criterion="gini", max_depth=5, random_state=42)

# Train model
model.fit(X_train, y_train)
```


4. Make Predictions

```
y_pred = model.predict(X_test)
```

5. Evaluate Model Performance

► Accuracy Score

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

6. Visualize the Decision Tree

```
from sklearn.tree import plot_tree

plt.figure(figsize=(12, 6))
plot_tree(model, filled=True, feature_names=X.columns, class_names=["No", "Yes"])
plt.show()
```

7. Optimize the Model (If Needed)

► Avoid Overfitting by setting

- `max_depth` (limit tree depth).
- `min_samples_split` (minimum samples needed to split a node).
- `min_samples_leaf` (minimum samples per leaf node).

```
model = DecisionTreeClassifier(max_depth=4, min_samples_split=5, min_samples_leaf=2)
model.fit(X_train, y_train)
```

Conclusion

- ✓ Decision Tree Classifiers are powerful for classification problems.
- ✓ They split data recursively based on the most informative feature.
- ✓ Hyperparameter tuning helps avoid overfitting.

Example

- ▶ Decision Tree Classification Example (Titanic Dataset)
- ▶ We'll use the **Titanic dataset** to predict whether a passenger survived or not using features like age, fare, and passenger class.

Step 1: Import Necessary Libraries



```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load and Explore the Dataset

```
# Load dataset
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Display first few rows
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Step 3: Data Preprocessing

- Drop unnecessary columns (PassengerId , Name , Ticket , Cabin).
- Handle missing values in **Age** and **Embarked**.
- Convert categorical variables (Sex , Embarked) into numerical format.

```
▶ # Drop unnecessary columns
df = df.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'])

# Fill missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Convert categorical variables
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True) # One-Hot Encoding
```

```
▶ # Display cleaned dataset
print(df.head())
```

```
⇒
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S
0	0	3	0	22.0	1	0	7.2500	False	True
1	1	1	1	38.0	1	0	71.2833	False	False
2	1	3	1	26.0	0	0	7.9250	False	True
3	1	1	1	35.0	1	0	53.1000	False	True
4	0	3	0	35.0	0	0	8.0500	False	True

Step 4: Split Data into Training and Testing Sets

```
# Define features and target variable
X = df.drop(columns=['Survived'])
y = df['Survived']

# Split data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 5: Train the Decision Tree Model

✓ 0s ▶ # Initialize and train model
model = DecisionTreeClassifier(criterion="gini", max_depth=4, random_state=42)
model.fit(X_train, y_train)



▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier(max_depth=4, random_state=42)

Step 6: Make Predictions

✓
s



```
y_pred = model.predict(X_test)
```

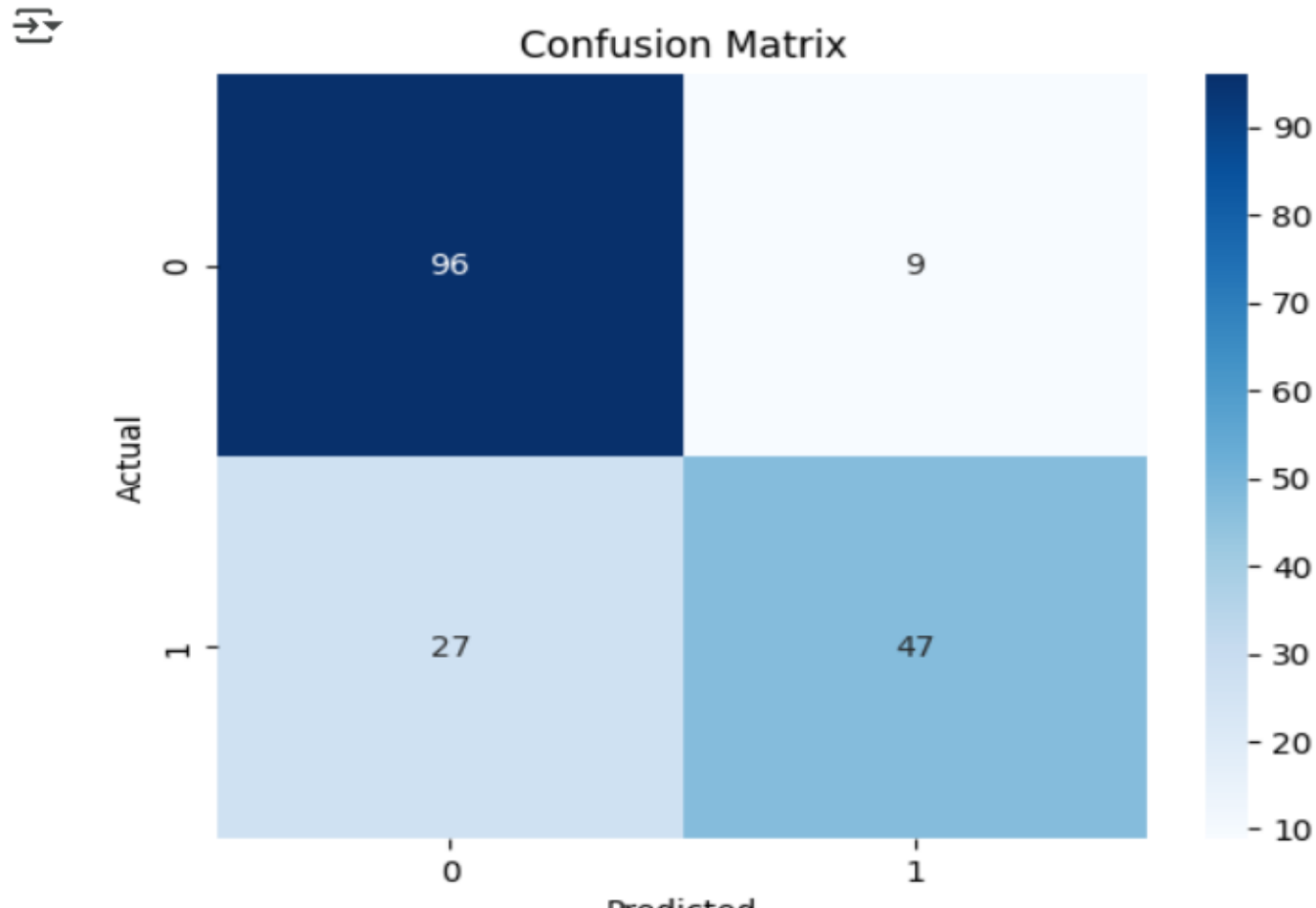
Step 7: Evaluate Model Performance

```
▶ accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy:", accuracy)
```

```
↔ Accuracy: 0.7988826815642458
```

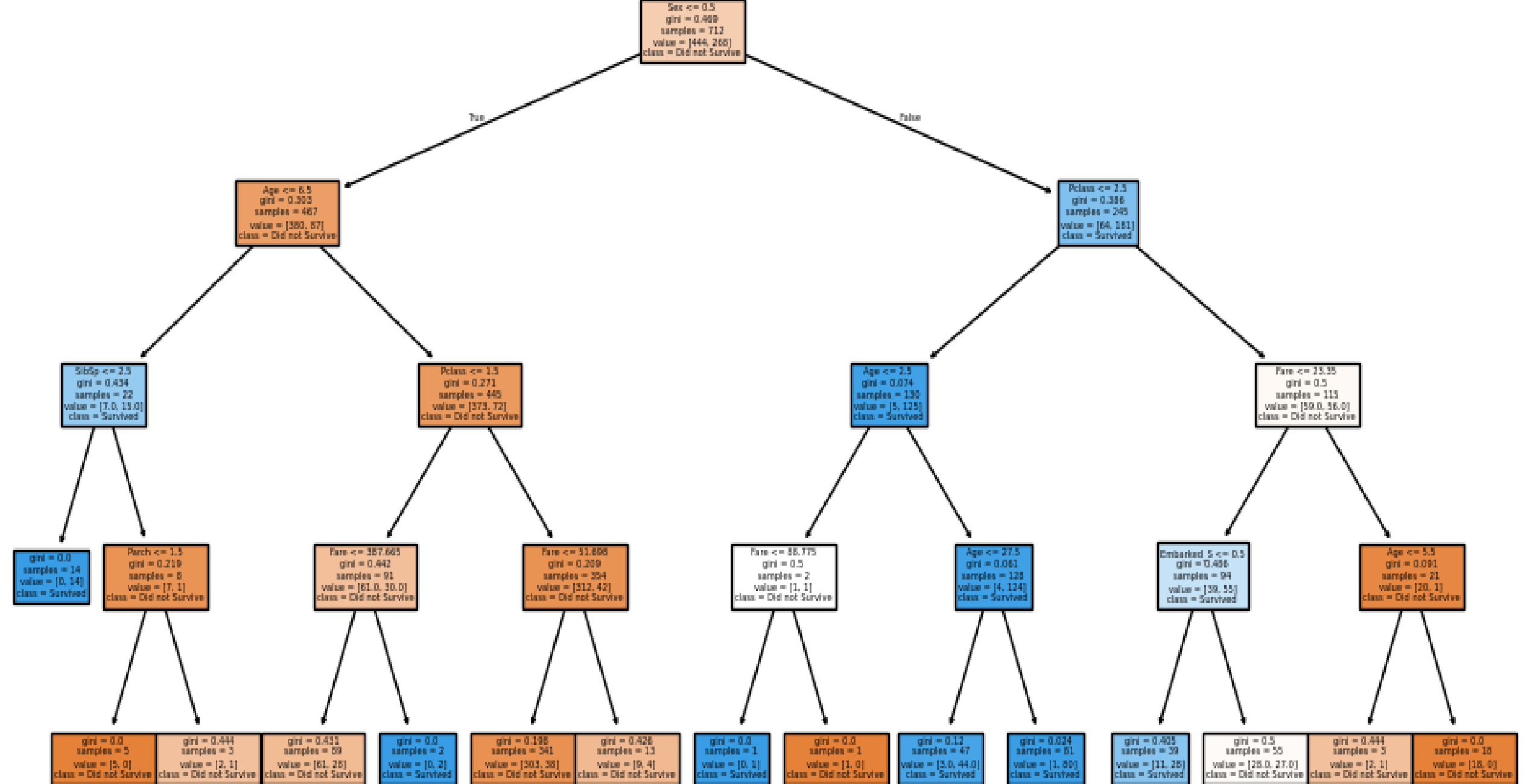
Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Step 8: Visualize the Decision Tree

```
▶ plt.figure(figsize=(12, 6))  
  plot_tree(model, filled=True, feature_names=X.columns, class_names=["Did not Survive", "Survived"])  
  plt.show()
```

Step 9: Improve the Model (Hyperparameter Tuning)

```
# Optimized Decision Tree
model_opt = DecisionTreeClassifier(max_depth=3, min_samples_split=5, min_samples_leaf=2, random_state=42)
model_opt.fit(X_train, y_train)

# Evaluate performance again
y_pred_opt = model_opt.predict(X_test)
accuracy_opt = accuracy_score(y_test, y_pred_opt)
print("Optimized Accuracy:", accuracy_opt)
```

```
➡ Optimized Accuracy: 0.7988826815642458
```

Bayes Classification Method - Naïve Bayes Classification

- ▶ “What are Bayesian classifiers?” Bayesian classifiers are statistical classifiers.
- ▶ They can predict class membership probabilities such as the probability that a given tuple belongs to a particular class.

Naïve Bayes Classification

- ▶ Naïve Bayes is a probabilistic machine learning algorithm used for classification tasks.
- ▶ It is based on Bayes' Theorem and assumes that features are independent of each other, which is why it is called "naïve."

Bayes' Theorem (Formula)

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Where:

- $P(A|B)$ → Probability of class **A** given evidence **B** (Posterior Probability)
- $P(B|A)$ → Probability of evidence **B** given class **A** (Likelihood)
- $P(A)$ → Probability of class **A** (Prior Probability)
- $P(B)$ → Probability of evidence **B** (Marginal Probability)

Types of Naïve Bayes Classifiers

- ▶ 1. Gaussian Naïve Bayes (GNB)
 - ▶ Used when features are **continuous** and follow a **normal distribution**.
 - ▶ Example: **Predicting whether a person has a disease based on age and blood pressure.**
- ▶ 2. Multinomial Naïve Bayes
 - ▶ Used for **discrete data** (e.g., word counts in text classification).
 - ▶ Example: **Spam email classification (word frequency).**
- ▶ 3. Bernoulli Naïve Bayes
 - ▶ Used for **binary features** (0 or 1).
 - ▶ Example: **Document classification (word presence/absence in a document).**

Naïve Bayes Example in Python (Spam Classification)

Step 1: Import Libraries

```
▶ import pandas as pd  
import numpy as np  
from sklearn.model_selection import train_test_split  
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.naive_bayes import MultinomialNB  
from sklearn.metrics import accuracy_score, classification_report
```


Step 2: Load Dataset

```
▶ # Load SMS spam dataset
url = "https://raw.githubusercontent.com/dD2405/Twitter_Sentiment_Analysis/master/train.csv"
df = pd.read_csv(url)[['label', 'tweet']] # Keeping only label & text

# Convert labels (0: Not Spam, 1: Spam)
df['label'] = df['label'].map({0: 'Not Spam', 1: 'Spam'})
```

Step 3: Preprocess Data

```
▶ # Split data into features (X) and target (y)
X = df['tweet']
y = df['label']

# Convert text into numerical features using CountVectorizer
vectorizer = CountVectorizer(stop_words='english')
X_vectorized = vectorizer.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test_size=0.2, random_state=42)
```

Step 4: Train Naïve Bayes Model

✓
0s



```
# Initialize and train Naïve Bayes classifier
```

```
nb_model = MultinomialNB()
```

```
nb_model.fit(X_train, y_train)
```



▼ MultinomialNB ⓘ ?

MultinomialNB()

Step 5: Make Predictions

```
▶ y_pred = nb_model.predict(X_test)
```

Step 6: Evaluate Model



```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Classification report
print(classification_report(y_test, y_pred))
```



Accuracy: 0.9440012513686845

	precision	recall	f1-score	support
Not Spam	0.97	0.97	0.97	5937
Spam	0.62	0.56	0.59	456
accuracy			0.94	6393
macro avg	0.79	0.77	0.78	6393
weighted avg	0.94	0.94	0.94	6393

When to Use Naïve Bayes?

- ▶ Works well with text data (Spam filtering, Sentiment Analysis, NLP)
- ▶ Fast and efficient, even for large datasets
- ▶ Performs well even with small datasets
- ▶ Not ideal when features are highly correlated

Deep Dive into Probability Calculations in Naïve Bayes

- ▶ Example: Spam Email Classification
- ▶ Let's say we want to classify an email as **Spam (S)** or **Not Spam (NS)** based on whether it contains specific words.

Example Training Data:

Email ID	Contains "Free"	Contains "Win"	Spam (1) / Not Spam (0)
1	Yes	Yes	Spam (1)
2	Yes	No	Spam (1)
3	No	Yes	Not Spam (0)
4	Yes	Yes	Not Spam (0)
5	No	No	Not Spam (0)

Now, we receive a **new email** that contains **"Free"** and **"Win"**. We need to classify it as **Spam (S)** or **Not Spam (NS)**.

Step-by-Step Probability Calculation

► Step 1: Calculate Prior Probabilities

- $P(S)$ = Probability of Spam

$$P(S) = \frac{\text{Spam Emails}}{\text{Total Emails}} = \frac{2}{5} = 0.4$$

- $P(NS)$ = Probability of Not Spam

$$P(NS) = \frac{\text{Not Spam Emails}}{\text{Total Emails}} = \frac{3}{5} = 0.6$$

Calculate Likelihood Probabilities

For Spam (S)

- $P(\textit{Free}|S)$ = Emails that are Spam and contain "Free" / Total Spam Emails

$$P(\textit{Free}|S) = \frac{2}{2} = 1$$

- $P(\textit{Win}|S)$ = Emails that are Spam and contain "Win" / Total Spam Emails

$$P(\textit{Win}|S) = \frac{1}{2} = 0.5$$

For Not Spam (NS)

- $P(Free|NS)$ = Emails that are Not Spam and contain "Free" / Total Not Spam Emails

$$P(Free|NS) = \frac{1}{3} \approx 0.33$$

- $P(Win|NS)$ = Emails that are Not Spam and contain "Win" / Total Not Spam Emails

$$P(Win|NS) = \frac{1}{3} \approx 0.33$$

Calculate Posterior Probability using Bayes' Theorem

For Spam (S)

$$P(S|Free, Win) = \frac{P(Free|S) \cdot P(Win|S) \cdot P(S)}{P(Free, Win)}$$

$$P(S|Free, Win) = \frac{(1) \cdot (0.5) \cdot (0.4)}{P(Free, Win)}$$

$$P(S|Free, Win) = \frac{0.2}{P(Free, Win)}$$

For Not Spam (NS)

$$P(NS|Free, Win) = \frac{P(Free|NS) \cdot P(Win|NS) \cdot P(NS)}{P(Free, Win)}$$

$$P(NS|Free, Win) = \frac{(0.33) \cdot (0.33) \cdot (0.6)}{P(Free, Win)}$$

$$P(NS|Free, Win) = \frac{0.065}{P(Free, Win)}$$

Compare Probabilities

Since $P(S|Free, Win) = 0.2$ is **greater** than $P(NS|Free, Win) = 0.065$, we classify the email as Spam. ✓

Installing Scikit-Learn in Python

- ▶ Scikit-Learn is a popular machine learning library in Python. You can install it using **pip** or **conda** (if using Anaconda).

✓
7s

▶ `pip install scikit-learn`



```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.5.0)
```

To upgrade to the latest version:

```
▶ pip install --upgrade scikit-learn
```

```
↔ Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (1.6.1)  
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.26.4)  
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.13.1)  
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.4.2)  
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.5.0)
```

To verify the installation:

✓
0s



```
import sklearn  
print(sklearn.__version__)
```



1.6.1

Question:

- ▶ What are different python libraries used for data analysis? [3]
- ▶ Explain data preprocessing in detail. [10]
- ▶ Differentiate between Predictive, Descriptive, and Prescriptive data analysis. [4]

OR

- ▶ What are association rules? Explain Apriori Algorithm in brief. [5]
- ▶ Write python program to apply the logistic regression model and check for goodness of fit. [7]
- ▶ What is the decisiontree? Explain with an example.

Question:

- ▶ What do you mean by Linear Regression? Elaborate the types. [6]
- ▶ Explain the Apriori algorithm with an example. [6]
- ▶ Write a short note on the following: [6]
 - i) FP growth
 - ii) Decision Tree Classification

OR

- ▶ Explain Data transformation using function and mapping. [6]
- ▶ Write a short note on the following: [6]
 - i) Removing duplicates from the data set.
 - ii) Handling missing data.
- ▶ Explain association rules with examples. [6]